Topic: Terraform Data Sources — Import vs Data Source, `data` Block, Existing Security Group → New EC2, AMI Lookup with Filters

(Your notes, cleaned up from dictation + screenshots — Lecture 7)

Quick recall

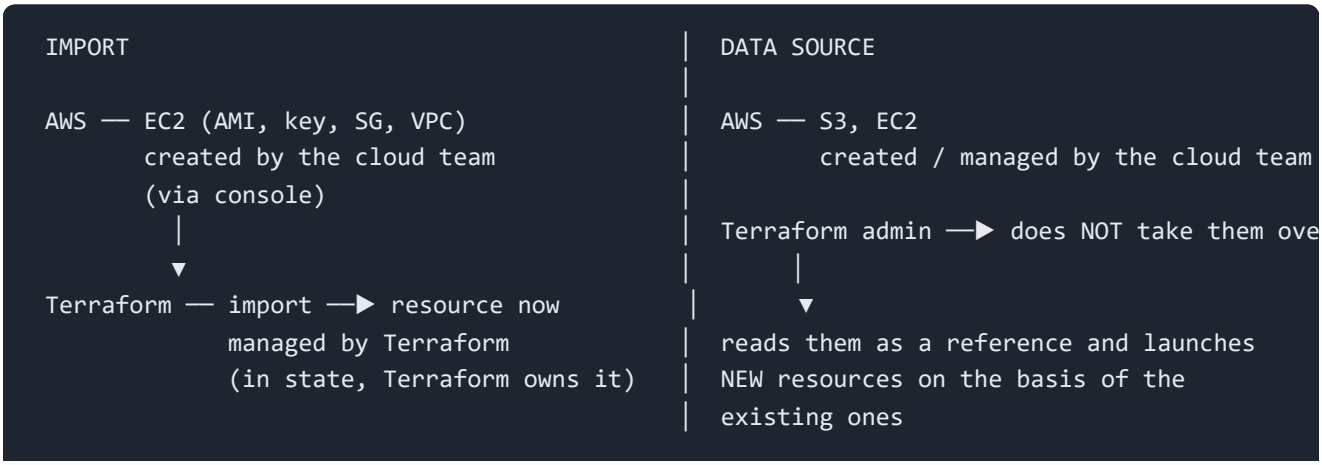
> **Definition:** In Terraform, a **data source** allows a configuration to **fetch or compute information defined outside of Terraform**, or from another distinct Terraform workspace. It serves as a **read-only view** of your infrastructure, enabling you to **reference existing resources without modifying or managing them**.

In one line: **a data source reads, a resource manages.** A `data` block never creates, changes or destroys anything. It only fetches details (ID, ARN, VPC, AMI…) of something that already exists, so other resources can use them.

---

1. Import vs Data Source: the whiteboard picture

Both deal with infrastructure that **already exists** in AWS, but they solve different problems.



Import: suppose we have AWS infrastructure created with the console, and later we decide to bring it **under Terraform**. We use `terraform import` (or an `import {}` block, see Chapter 03). After that, Terraform **owns** the resource. It's in the state, and `plan` / `apply` / `destroy` act on it.

Data source: suppose AWS has an S3 bucket, EC2 instances and so on, all **managed by the cloud team**. The Terraform admin **doesn't import** those resources under Terraform. They **take a reference from the currently running resources** and **launch new resources on the basis of the existing ones**. The existing resource stays with the cloud team, and Terraform only reads it.

	`terraform import`	Data source (`data` block)
Purpose	Bring an existing resource **under Terraform management**	**Read** details of an existing resource to use elsewhere
Who owns it afterwards	Terraform	Still whoever created it (cloud team / console / another TF project)
Can Terraform change/destroy it?	**Yes**, `destroy` would delete it	**No**, read-only
Block used	`resource` (+ `import {}` or CLI import)	`data`
Typical use	Adopting console-built infra into IaC	Using a shared VPC, SG, AMI, subnet, key, certificate… as input

> **Every resource has a resource block as well as a data source block.** For example, the AWS provider has `resource "aws_security_group"` **and** `data "aws_security_group"`, `resource "aws_instance"` **and** `data "aws_instance"`, `resource "aws_vpc"` **and** `data "aws_vpc"`, and so on. *(Nearly all of them do. A few data sources have no matching resource, e.g.* `aws_ami_ids`, `aws_caller_identity`, `aws_availability_zones`.*)*

---

2. Data source block syntax

```
data "aws_security_group" "example" {      ──▶ data  "<provider_type>"  "<local name>"
  name = "ram_sg"                          ──▶ arguments = the search/filter criteria
}

data.aws_security_group.example.id         ──▶ how you reference it:
                                               data.<TYPE>.<NAME>.<ATTRIBUTE>
```

- The `data` keyword plays the same role as `resource`, but it **reads** instead of creating.
- The arguments inside are **lookup criteria** (name, tags, filters, ID…), not settings to apply.
- References always start with `data.`, which is the difference from a resource reference (`aws_instance.example.id`).

- If the lookup finds **nothing** (or, for single-result data sources, **more than one match**), `plan` fails with an error. That's good, because you find out before anything is built.

---

3. Lab 1: read an existing security group

In the AWS console (N. Virginia, us-east-1) there's already a security group created outside Terraform:

Name	Security group ID	Security group name	VPC ID	Description
ram_SG	`sg-060a09f2f6b974ced`	`ram_sg`	`vpc-2ab1794d`	for demo

Suppose this **ram security group already exists** on AWS. Now we will launch a new resource on the basis of it. First, just read it and print its ID:

```
# main.tf
provider "aws" {
  region = "us-east-1"
}

# Look up an existing security group by name in NV region
data "aws_security_group" "example" {
  name = "ram_sg"
}

output "my-ouput" {
  value = data.aws_security_group.example.id
}
```

```
[root@Devops-server project1]# terraform apply --auto-approve
data.aws_security_group.example: Reading...
data.aws_security_group.example: Read complete after 2s [id=sg-060a09f2f6b974ced]

Changes to Outputs:
  + my-ouput = "sg-060a09f2f6b974ced"

You can apply this plan to save these new output values to the Terraform state, without char

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

my-ouput = "sg-060a09f2f6b974ced"
```

What this shows: when we run `terraform apply`, it **won't launch any new resource**. It gets the information from the existing one:

- `Reading...` → `Read complete` is the data source talking to the AWS API.
- `0 added, 0 changed, 0 destroyed`, so nothing was created.
- The output matches the console exactly: `sg-060a09f2f6b974ced`.

> 📝 The `name` argument matches the **Security group name** column (`ram_sg`), not the **Name tag** (`ram_SG`). Those are two different fields in AWS. To match on the Name tag instead, use `tags = { Name = "ram_SG" }` or a `filter { name = "tag:Name" ... }`.

4. Lab 2: use that data to create a new EC2 instance

Now we've got the data from the existing resource. With it we create a new resource: an EC2 instance attached to the **existing** security group.

```
# Look up an existing security group by name in NV region
data "aws_security_group" "example" {
  name = "ram_sg"
}

output "my-ouput" {
  value = data.aws_security_group.example.id
}

# Launch an EC2 instance and associate it with the existing security group in NV region
resource "aws_instance" "example" {
  ami                    = "ami-05ffe3c48a9991133"           # Replace with a real AMI ID
  instance_type          = "t2.micro"
  key_name               = "ram_nv"
  vpc_security_group_ids = [data.aws_security_group.example.id]
  tags = {
    Name   = "NV_server"
    source = "terraform"
  }
}
```

The key line is `vpc_security_group_ids = [data.aws_security_group.example.id]`:

- It's a **list**, hence the `[ ]`, because an instance can have several SGs.
- Terraform reads the SG first, then creates the instance. The reference gives an **implicit dependency**, just like resource-to-resource references in Chapter 02.

- `terraform destroy` removes **only** `NV_server`. `ram_sg` isn't in Terraform's managed resources, so it stays untouched.

---

5. Lab 3: look up the AMI instead of hard-coding it

A hard-coded AMI ID (`ami-05ffe3c48a9991133`) has two problems. It's **different in every region**, and it **goes stale** when a newer image comes out. The `aws_ami` data source finds the right one **dynamically**.

Reference example: latest Ubuntu 22.04 from Canonical

```
data "aws_ami" "ubuntu" {
  most_recent = true

  filter {
    name   = "name"
    values = ["ubuntu/images/hvm-ssd/ubuntu-jammy-22.04-amd64-server-*"]
  }

  filter {
    name   = "virtualization-type"
    values = ["hvm"]
  }

  owners = ["099720109477"] # Canonical
}

resource "aws_instance" "example" {
  ami           = data.aws_ami.ubuntu.id
  instance_type = "t3.micro"

  tags = {
    Name = "HelloWorld"
  }
}
```

Lab: latest Amazon Linux 2 AMI (`data.tf`)

```
# Define a data source to fetch the latest Amazon Linux 2 AMI
data "aws_ami" "latest_amazon_linux" {
  most_recent = true
  owners      = ["amazon"]
  filter {
    name   = "name"
    values = ["amzn2-ami-hvm-*-x86_64-gp2"]
  }
}

output "my-ouput" {
  value = data.aws_ami.latest_amazon_linux.id
}
```

How the arguments work together:

Argument	Meaning
`owners`	**Whose** images to search. `"amazon"` is an alias for Amazon's own account. Canonical (Ubuntu) is account `099720109477` . **Always set this.** Without it, anyone could publish a public AMI with a matching name.
`filter { name = "name" ... }`	Match the AMI **name** with wildcards (`*`). This is the same as the `--filters` option of `aws ec2 describe-images`
`filter { name = "virtualization-type" ... }`	Any other describe-images filter can be added: `architecture` , `root-device-type` , etc.
`most_recent = true`	Several AMIs match the wildcard. Pick the **newest** one (by creation date)

Then use it in a resource: `ami = data.aws_ami.latest_amazon_linux.id` .

> ⚠️ **Same output name in two files:** `main.tf` and `data.tf` both declare `output "my-ouput"` . If both files are in `project1` at the same time, Terraform fails with **"Duplicate output definition"**, because it reads every `.tf` in the folder together. Give each output a unique, meaningful name (e.g. `sg_id` , `ami_id`). The `ouput` spelling is just a label, so it works, but fix it in real code.

> ⚠️ **Amazon Linux 2 reached end of support on 30 June 2026.** For new servers, use Amazon Linux 2023: `values = ["al2023-ami-2023.*-x86_64"]` with `owners = ["amazon"]` .

---

6. How data sources behave (what's worth knowing beyond the lab)

- **When they're read:** at every `plan` / `apply` (during refresh). So the value is always **current**, not a snapshot from the first run.
- **They go into the state file** (as `"mode": "data"`), but Terraform **doesn't manage** them. `terraform state list` shows them as `data.aws_security_group.example`.
- `most_recent = true` **side effect:** when AWS publishes a newer AMI, the next `plan` shows the EC2 instance **must be replaced** (the AMI changed). In production, either pin the AMI or add `lifecycle { ignore_changes = [ami] }` and roll AMIs deliberately.
- **Values not known until apply:** if a data source's arguments depend on a resource that doesn't exist yet, the read is deferred to apply time, and its attributes show as `(known after apply)` in the plan.
- **Other workspaces / projects:** the "from another distinct Terraform workspace" part of the definition is the `terraform_remote_state` data source. It reads the **outputs** of another Terraform project's state (e.g. the network team's `vpc_id`).

Commonly used AWS data sources

Data source	Typical use
`aws_ami`	Latest AMI by name/owner filter
`aws_security_group`	Attach a shared SG to new instances
`aws_vpc` / `aws_subnets`	Deploy into a VPC built by the network team
`aws_availability_zones`	Spread resources across the region's AZs
`aws_caller_identity`	Current account ID (for ARNs, bucket names)
`aws_region`	Current region name
`aws_iam_policy_document`	Build IAM policy JSON in HCL
`aws_key_pair`	Check/reuse an existing key pair
`terraform_remote_state`	Read another Terraform project's outputs

Command reference (this chapter)

```
terraform plan                                  # data sources are read here: "Reading..." A
terraform apply --auto-approve                  # data-only config → 0 added, 0 changed, 0 d
terraform output                                # show the value read from the existing resc
terraform state list                            # data sources appear as data.<type>.<name>
terraform state show data.aws_security_group.example   # every attribute the lookup returned
terraform console                               # try: data.aws_ami.latest_amazon_linux.id
aws ec2 describe-images --owners amazon \
  --filters "Name=name,Values=amzn2-ami-hvm-*-x86_64-gp2" \
  --query 'sort_by(Images,&CreationDate)[-1].ImageId'   # same lookup from the AWS CLI, to c
```

Still to come

- `terraform_remote_state`: sharing outputs between Terraform projects
- Data sources inside **modules** (e.g. looking up the default VPC and subnets)
- `aws_iam_policy_document` for writing IAM policies in HCL